

Laboratory 1: Neural Models

Learning, Depth, Activations, and Output Layers

Eshan Saikia
2023A7PS1024G

The accompanying `neural_models.ipynb` contains the executable implementation and raw experimental evidence. This report records the problem formulation, design decisions, measured results, and reflections.

1 Problem Specification and Design

The task is to raise a warning exactly when two binary sensors disagree. Thus

$$X = \{0, 1\}^2, \quad Y = \{0, 1\},$$

with targets 0, 1, 1, 0 for inputs (0, 0), (0, 1), (1, 0), (1, 1) respectively.

A single affine transformation followed by a sigmoid still has a linear decision boundary. Since the two classes occupy opposite corners of the input square, it cannot represent XOR perfectly.

Pre-execution prediction. An affine-only classifier should reduce the loss but should not classify all four XOR examples correctly at the same time.

Baseline design. A $2 \rightarrow 2 \rightarrow 1$ network with sigmoid hidden units and one output logit, trained with `BCEWithLogitsLoss` and gradient-based optimization. We use random Xavier weight initialization. Sigmoid maps the binary output logit to a probability, while binary cross-entropy measures error against the binary target; the PyTorch logits form combines these stably.

Learning is validated using loss reduction, correct classification of all four examples, and a nonzero first-layer gradient during a successful run.

2 LLM-Assisted Implementation

The implementation request used in the LLM-assisted workflow was:

Generate minimal PyTorch code for XOR with inputs `[[0,0],[0,1],[1,0],[1,1]]` and targets `[[0],[1],[1],[0]]`. Implement exactly a 2-input $\rightarrow$ 2-hidden-unit $\rightarrow$ 1-output network, use a sigmoid hidden activation, random weight initialization, one output logit with `BCEWithLogitsLoss`, full-batch training for a few thousand CPU steps, and a gradient-based optimizer. Report initial/final loss, four sigmoid probabilities, thresholded labels, and one first-layer gradient tensor after `backward()`. Set a seed for reproducibility. Make the forward pass, loss, backward pass, and optimizer update easy to identify. Do not change the architecture or task.

Two engineering changes were made during verification: (1) Xavier initialization and the random seed were made explicit for reproducibility; (2) diagnostic outputs were added for gradients, loss history, symmetry, activations, and predictions. The scientific task and architecture were unchanged.

The source code was then inspected to verify the requested sequence: forward pass, scalar loss, `backward()`, and optimizer update. Properties such as tensor shapes and architecture can be checked statically; loss reduction, predictions, gradients, and symmetry require execution.

3 Experimental Results

3.1 Binary XOR and Backpropagation

Input	Target	Probability	Prediction
(0, 0)	0	0.000680889	0
(0, 1)	1	0.999374092	1
(1, 0)	1	0.999157548	1
(1, 1)	0	0.000625538	0

Initial loss: 0.710871339.

Final loss: 0.000693891.

All four labels were classified correctly.

At training step 50, the first-layer gradient was approximately

$$\nabla_{W^{(1)}} L \approx \begin{bmatrix} 0.0069 & -0.0073 \\ 0.0028 & -0.0021 \end{bmatrix}, \quad \|\nabla_{W^{(1)}} L\|_2 = 0.010602132.$$

Because the loss uses mean reduction over the four examples, this batch gradient is the average of the corresponding example-wise gradients.

3.2 Zero Initialization and Symmetry

With all trainable parameters initialized to zero, the two hidden-layer rows remained exactly identical at steps 1, 2, 3, 5, and 10. The first-layer gradient at step 1 was also zero. Identical hidden units therefore receive identical updates and cannot specialize into distinct features.

3.3 Activation Experiment

Hidden activation	Final loss	4/4 correct	Early $\ \nabla W^{(1)} L\ _2$
Sigmoid	0.000694	Yes	0.010602
Tanh	0.000222	Yes	0.074817
ReLU	0.000085	Yes	0.130315

These are observations for the four-point experiment, not a universal ranking of activation functions. Sigmoid and tanh can attenuate gradients near saturation, while ReLU has derivative 0 on its negative branch and 1 on its positive branch. The accompanying pre-activation outputs allow these mechanisms to be distinguished directly.

3.4 Three-Class Extension

The labels become $[0, 1, 1, 2]$, with class 0 for (0, 0), class 1 for disagreement, and class 2 for (1, 1). The hidden layer remains size 2 and the output layer maps the two hidden units to 3 logits; in PyTorch the final weight tensor has shape (3, 2).

Modification prompt.

Modify the previous 2-input $\rightarrow$ 2-hidden-unit PyTorch model only in the output/loss portion for labels $[0, 1, 1, 2]$. Keep the hidden representation and activation unchanged. Replace the single output with three logits and use CrossEntropyLoss. Report the final loss, class probabilities for all four inputs, predicted classes, a probability-sum check, and a numerical verification of the $p - y$ logit-gradient identity. Do not change the hidden architecture or use a pretrained model.

Before execution, the expected final weight shape was (3, 2), each example was expected to produce three logits, softmax probabilities were expected to sum to one, and the cross-entropy logit gradient was expected to equal $p - y$.

Input	Target	$P(0)$	$P(1)$	$P(2)$
(0, 0)	0	0.999880075	0.000119899	0.000000026
(0, 1)	1	0.000074029	0.999853492	0.000072516
(1, 0)	1	0.000067527	0.999841332	0.000091054
(1, 1)	2	0.000001178	0.000257353	0.999741495

Initial loss: 1.076565385. Final loss: 0.000170900. All four classes were predicted correctly.

For (0, 1), the softmax probabilities summed to 1.000000000000. Adding 100 to all logits changed the probability vector by only 2.76×10^{-10} . The autograd logit gradient matched $p - y$ with maximum absolute error 2.91×10^{-11} .

Subtracting the maximum logit before exponentiation is therefore numerically safe because adding or subtracting the same constant from every logit leaves softmax probabilities unchanged.

4 Reflection Questions

1. **Depth vs. nonlinearity.** XOR shows that affine-only depth still represents an affine function. A nonlinear hidden layer changes the representational class enough to model XOR.
2. **Useful backpropagation signal.** A nonzero gradient alone is weak evidence. Here, gradient-based updates substantially reduced the loss and produced all four correct labels, showing that the learning signal was useful.
3. **Identical/zero initialization.** Symmetric hidden units compute identical features and receive identical gradients, so the updates preserve symmetry and the units cannot spontaneously specialize.
4. **Activation and gradient.** Sigmoid and tanh can attenuate gradients near saturation; ReLU gives derivative 0 for negative pre-activations and 1 for positive ones. The measured norms are specific to this run.
5. **Output/loss pairing.** The output representation and loss must match the task: a binary problem uses a binary objective, whereas mutually exclusive classes use multiple logits with multiclass cross-entropy.
6. **LLM productivity and verification.** The LLM reduced implementation effort by translating the specified architecture into PyTorch and helping expose diagnostics. Human verification remained necessary for architecture, initialization behaviour, predictions, gradients, and numerical invariants.
7. **Tests at larger scale.** Loss and prediction checks, shape checks, reproducibility, gradient sanity checks, and probability invariants remain useful. Exhaustive finite-difference gradient checks become expensive with parameter count and should be replaced by targeted or sampled checks.

5 Think About It

Representation. XOR distinguishes having parameters from having the right representation.

Hidden units. Backpropagation determines useful hidden features through the loss; the units have no separate targets.

Activation diagnosis. Inspect pre-activations and activations to distinguish sigmoid/tanh saturation from inactive ReLU units.

Modern language models. Next-token prediction still uses a conditional probability distribution, but the vocabulary, learned representation, context, and model scale are vastly larger.